

Pizza Maestro

Nápolyi pizzatészta-hozzávaló kalkulátor

Szoftverdokumentáció

Vizsgaremek — Programfejlesztő és tesztelő felnőttképzés

Készítette: Kurecskó Cintia

GitHub: github.com/kurecskocintia/VizsgaRemek

2026. szeptember

Technológiák: React · Vite · ASP.NET Core Web API · .NET 8 · MySQL · Entity Framework Core · JWT
· Axios

Tartalomjegyzék

1. Bevezetés	4
1.1 A dokumentum célja	4
1.2 A projekt háttere	4
2. A projekt áttekintése	5
2.1 Célkitűzések	5
2.2 Célközönség és felhasználási területek	5
2.3 Felhasznált technológiák	5
3. Rendszerarchitektúra	6
3.1 Kliens–szerver felépítés	6
3.2 Mappastruktúra	6
3.3 A kérés életútja	6
4. Adatbázis-terv	7
4.1 Entitások	7
4.2 Kapcsolatok	7
4.3 EF Core és MySQL	7
5. Felhasználók és hitelesítés	8
5.1 Regisztráció	8
5.2 Bejelentkezés és JWT	8
5.3 Saját receptek védelme	8
6. Funkcionális bemutatás	9
6.1 Kezdőoldal és navigáció	10
6.2 A pizzatészta-kalkulátor	11
6.3 Eredmény és AVPN-jelzés	12
6.4 Mentett receptek	13
6.5 Reszponzív és felhasználói működés	14
7. Backend implementáció részletei	15
7.1 REST API-végpontok	15
7.2 Tésztaszámítási algoritmus	16
7.3 Élesztő- és fermentációs becslés	16
7.4 Receptkezelés	17
7.5 Frontend–backend kommunikáció	17
8. Biztonsági megoldások	18
8.1 Jelszókezelés	18
8.2 JWT-alapú jogosultság	18
8.3 Tulajdonosi ellenőrzés	18
8.4 Éles üzemeltetéshez szükséges megerősítések	18
9. Tesztelés	19
9.1 Tesztelési stratégia	19

9.2 Manuális teszteset-napló.....	19
9.3 Fejlesztési és hibajavítási folyamat.....	20
10. Telepítési és üzemeltetési útmutató.....	21
10.1 Előfeltételek	21
10.2 Backend indítása	21
10.3 Frontend indítása.....	21
10.4 Adatbázis konfiguráció	21
11. Követelmények áttekintése	22
12. Továbbfejlesztési lehetőségek	23
12.1 Automatizált tesztek	23
12.2 Konfiguráció és titkok	23
12.3 Kalkuláció továbbfejlesztése	23
12.4 Felhasználói funkciók	23
13. Kódminőség és karbantarthatóság	24
14. Összegzés	25
Forrásjegyzék	26

1. Bevezetés

1.1 A dokumentum célja

Ez a dokumentum a Pizza Maestro nevű webalkalmazás szoftverdokumentációja, amely a Programfejlesztő és tesztelő felnőttképzés vizsgaremekeként készült. Célja, hogy a vizsgabizottság és a későbbi fejlesztő számára egyaránt áttekinthetően bemutassa a rendszer célját, felépítését, működését, adatkezelését, API-ját, biztonsági megoldásait, tesztelését és telepítését.

A dokumentáció alapját a jelenlegi GitHub repository, a forráskód, a projekt README-je, valamint a fejlesztés során végzett hibakeresések és módosítások adják. A GitHub repository jelenleg React + Vite frontendet, ASP.NET Core Web API (.NET 8) backendet és MySQL + Entity Framework Core adatkezelést tartalmaz.

1.2 A projekt háttere

A Pizza Maestro célja egy olyan, böngészőből használható nápolyi pizzatészta-kalkulátor létrehozása, amely nem pusztán egy egyszerű százalékszámítást végez, hanem figyelembe veszi a gombócok számát és tömegét, a hidratációt, a sótartalmat, az élesztő vagy kovász típusát, valamint a szobahőmérsékleti és hűtős fermentáció idő- és hőmérsékleti adatait.

A projekt fejlesztése során a működő alapfunkciók mellett több körben történt konfigurációs és integrációs javítás: kialakult a .NET 8-as backend, az EF Core és Pomelo MySQL kapcsolat, a Swagger dokumentáció, a JWT-alapú hitelesítés, a React/Vite kliens és az Axios kommunikáció. A fejlesztés során külön figyelmet kapott a localhostos HTTPS, a CORS, az API-cím helyes beállítása és a csomagfüggőségek verzióinak összehangolása.

2. A projekt áttekintése

2.1 Célkitűzések

- nápolyi pizzatészta receptjének kiszámítása megadott paraméterekből;
- friss és szárított élesztő, valamint folyékony kovász és lievito madre kezelése;
- fermentációs idő- és hőmérsékleti adatok kezelése;
- az AVPN-hez igazított hidratációs és sótartomány jelzése;
- regisztráció és bejelentkezés JWT-alapú hitelesítéssel;
- a kiszámított receptek személyhez kötött mentése, listázása és törlése;
- áttekinthető, responszív React felület biztosítása.

2.2 Célközönség és felhasználási területek

A rendszer elsősorban otthoni pizzakészítők és a nápolyi pizzakészítés iránt érdeklődő felhasználók számára készült. Használható recepttervezéshez, különböző fermentációs forgatókönyvek összehasonlításához és saját receptgyűjtemény kialakításához.

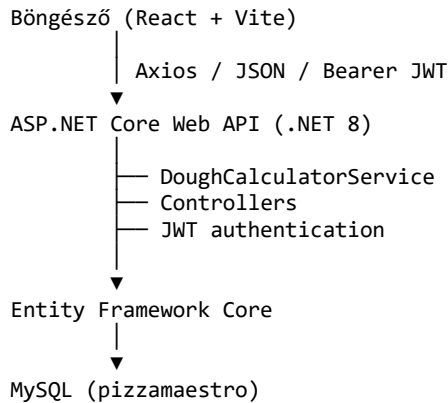
2.3 Felhasznált technológiák

Réteg	Technológia	Szerep
Frontend	React 19 + Vite 7	Komponensalapú felhasználói felület és fejlesztői szerver
HTTP kliens	Axios	REST API hívások és JWT fejléc kezelése
Backend	ASP.NET Core Web API (.NET 8)	Üzleti logika és REST végpontok
Adatkezelés	Entity Framework Core 8	ORM és adatbázis-leképezés
Adatbázis	MySQL	Felhasználók és receptek tartós tárolása
Hitelesítés	JWT Bearer	Bejelentkezési állapot és API-védelem
Dokumentáció	Swagger / Swashbuckle 6.6.2	API kipróbálása és dokumentálása

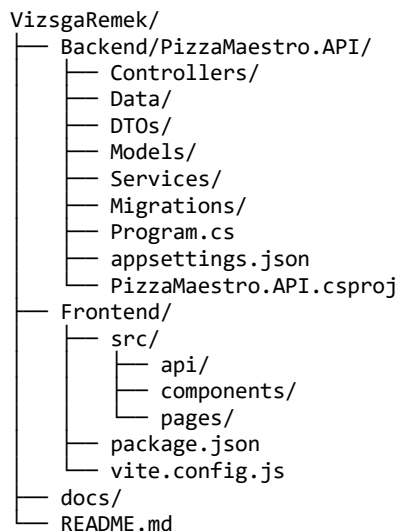
3. Rendszerarchitektúra

3.1 Kliens–szerver felépítés

A Pizza Maestro klasszikus kliens–szerver architektúrát követ. A böngészőben futó React alkalmazás Axioson keresztül JSON formátumú HTTP-kéréseket küld az ASP.NET Core Web API felé. A backend végzi a bemenetek ellenőrzését, a kalkulációt, a hitelesítést és az adatbázis-műveleteket. A kliens közvetlenül nem kommunikál a MySQL adatbázissal.



3.2 Mappastruktúra



3.3 A kérés életútja

Egy kalkuláció során a CalculatorPage összegyűjti az űrlap értékeit, a calculatorApi a /api/calculator végpontra küldi azokat, a CalculatorController ellenőrzi az alapvető bemeneteket, majd a DoughCalculatorService kiszámítja az összetevőket és az AVPN-jelzőket. A CalculatorResponse JSON-ként visszakérül a frontendhez, amely a ResultPanel komponensben jeleníti meg.

4. Adatbázis-terv

4.1 Entitások

Entitás	Fő mezők	Szerep
User	Id, UserName, Email, PasswordHash, CreatedAt	Felhasználói fiók
Recipe	Id, Name, YeastType, Balls, BallWeight	Mentett recept
Recipe → User	UserId	Tulajdonoshoz kötés

A jelenlegi modell két fő entitást használ. Az ApplicationDbContext a Users és Recipes DbSeteket definiálja, és egyedi indexet hoz létre az e-mail-címre és a felhasználónévre.

4.2 Kapcsolatok

Egy felhasználóhoz több recept tartozhat, ezért a kapcsolat 1:N. A Recipe.UserId idegen kulcs a User entitásra mutat. A törléshez Cascade szabály tartozik, így a felhasználóhoz tartozó receptek a felhasználó törlésekor automatikusan eltávolíthatók.

```
b.Entity<Recipe>()
    .HasOne(x => x.User)
    .WithMany(x => x.Recipes)
    .HasForeignKey(x => x.UserId)
    .OnDelete(DeleteBehavior.Cascade);
```

4.3 EF Core és MySQL

A backend az Entity Framework Core 8-at használja MySQL adatbázissal, a Pomelo.EntityFrameworkCore.MySql 8.0.3 provider segítségével. A projektben az adatbázis-kapcsolat és a modellleképezés az ApplicationDbContextben történik. A csomagverziókat tudatosan .NET 8-kompatibilis állapotban tartottuk.

A fejlesztés során az EF parancssori eszköz verziójának összehangolása is szükséges volt: a korábban használt 10.x-es dotnet-ef helyett 8.0.19-es verzióra történt visszaállítás, mert a projekt .NET/EF Core 8-as csomagokra épül.

5. Felhasználók és hitelesítés

5.1 Regisztráció

A regisztráció során a backend ellenőrzi a felhasználónév, e-mail és jelszó meglétét, legalább hat karakteres jelszót vár el, normalizálja az e-mail-címet, majd az ASP.NET Core Identity PasswordHasher<User> osztályával hash-eli a jelszót. A kliens soha nem kapja vissza a jelszó hashét.

5.2 Bejelentkezés és JWT

Sikeres bejelentkezés után a backend JWT tokenet generál. A tokenben a felhasználó azonosítója, neve és e-mail-címe claimként szerepel. A token lejárat ideje konfigurálható, alapértelmezettként 1440 perc. A backend a JWT aláírását, issuerét, audience értékét és érvényességi idejét ellenőrzi.

```
POST /api/auth/register  
POST /api/auth/login
```

```
Authorization: Bearer <JWT>
```

5.3 Saját receptek védelme

A RecipesController teljes egészében [Authorize] alatt működik. A user azonosítója a JWT Namelfidentifier claimjéből származik, ezért a kliens nem adhat meg szabadon másik felhasználó-azonosítót. A lekérdezés minden esetben a bejelentkezett felhasználóhoz tartozó recepteket szűri.

6. Funkcionális bemutatás

A Pizza Maestro fő felhasználói folyamata öt jól elkülöníthető területből áll: kezdőoldal, regisztráció/bejelentkezés, kalkulátor, mentett receptek és kijelentkezés. A React App komponens egyszerű állapot alapján váltja az oldalakat, külön router csomag nélkül.

Funkció	Elérhetőség	Leírás
Kezdőoldal	Mindenki	A rendszer rövid bemutatása és navigáció
Regisztráció	Mindenki	Új felhasználói fiók létrehozása
Bejelentkezés	Mindenki	JWT token kérése
Kalkulátor	Mindenki	Tésztaösszetevők számítása
Recept mentése	Bejelentkezett	Kiszámított recept adatbázisba mentése
Mentett receptek	Bejelentkezett	Saját receptek listázása és törlése
Kijelentkezés	Bejelentkezett	Token és lokális felhasználói állapot törlése

A dokumentáció további oldalai a működés egyes részterületeit külön mutatják be. A funkcionális fejezet szándékosan nem csak a felületet, hanem az ahhoz tartozó backend folyamatot is ismerteti.

6.1 Kezdőoldal és navigáció

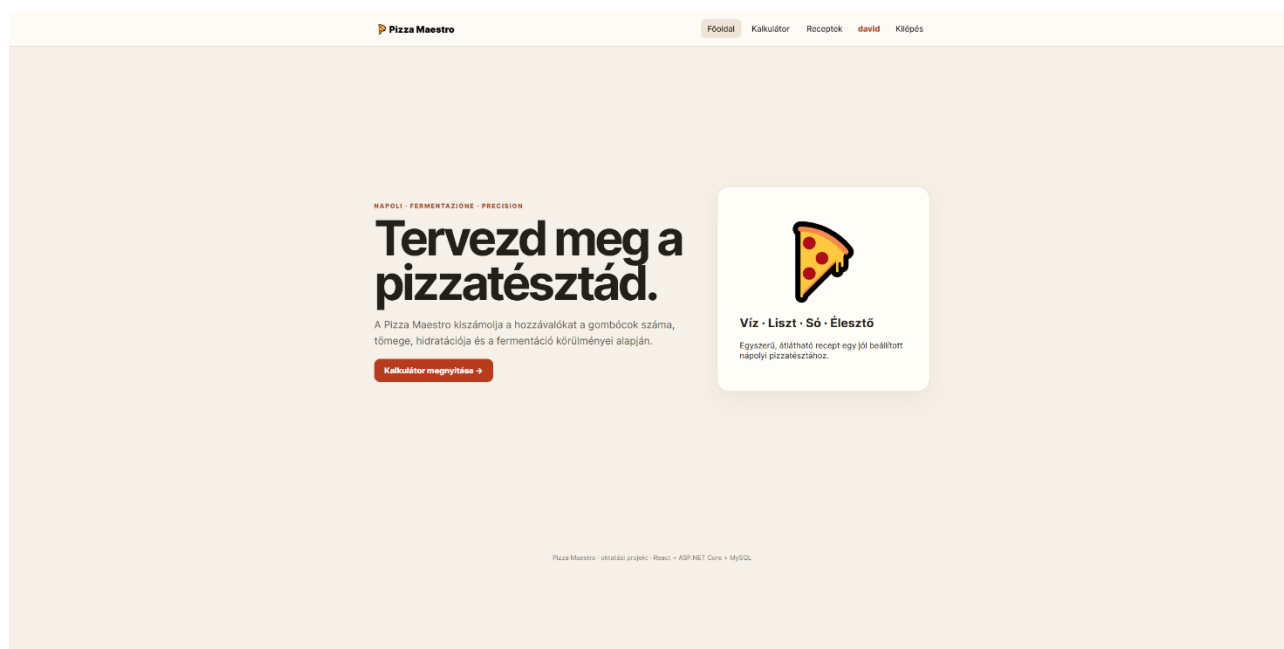
Az App.jsx központi komponens kezeli az aktuális oldalt és a bejelentkezett felhasználót. A navigációt a Navbar komponens biztosítja, az oldal tartalmát pedig a page állapot értéke alapján rendereli a rendszer.

```
const [page, setPage] = useState("home");
const [user, setUser] = useState(getCurrentUser());

{page === "home" && <HomePage go={setPage} />}
{page === "calculator" && <CalculatorPage user={user} go={setPage} />}
{page === "recipes" && <RecipesPage user={user} go={setPage} />}
```

A felület használati logikája egyszerű: a látogató először a kezdőoldalt látja, innen a kalkulátor vagy a hitelesítési felület érhető el. Bejelentkezés után a kliens localStorage-ban tárolja a JWT-t és a felhasználó alapadatait.

Kijelentkezéskor a localStorage-ból törlődik a pizzaMaestroToken és a pizzaMaestroUser kulcs, majd a React állapot visszaáll a kezdőoldalra. Ez a frontend oldali munkamenet lezárását biztosítja.



1. ábra Kezdőlap

6.2 A pizzatészta-kalkulátor

A kalkulátor központi felülete a CalculatorPage és a DoughForm komponens együttműködésével működik. Az űrlap többek között a gombócok számát, a gombóc tömegét, a hidratációt, a sót, az élesztő/kovász típusát, valamint a szobahőmérsékleti és hűtős fermentáció paramétereit fogadja.

Bemenet	Tartomány / példa	Jelentés
Gombócok száma	1–30	Elkészítendő tésztagombócok száma
Gombóc tömege	100–500 g	Egy gombóc célzott tömege
Hidratáció	45–90%	Víz/liszt arány
Só	0–6%	Só/liszt arány
Élesztő/kovász	4 típus	Friss, szárított, li.co.li., lievito madre
Szobahőmérséklet	10–35 °C	Első fermentációs szakasz
Szobahőmérsékleti idő	0–72 óra	Első szakasz időtartama
Hűtő	1–12 °C	Hideg fermentáció
Hűtős idő	0–168 óra	Második szakasz időtartama

Az alapértelmezett értékek egy 4 × 250 grammos, 62%-os hidratációjú, 2,8%-os sótartalmú, 6 órás szobahőmérsékleti és 18 órás hűtős fermentációs példát adnak.

2. ábra Receptkalkuláció bemeneti feltételei

6.3 Eredmény és AVPN-jelzés

A számítás eredménye a ResultPanel komponensben jelenik meg. A panel megmutatja az össztészta tömegét, a liszt, víz, só és élesztő/kovász mennyiségét, a hidratációt, a sótartalmat és a fermentációt. Kovász használata esetén külön feltünteti a kovászban lévő liszt- és vízmennyiséget is.

A rendszer külön jelzi, hogy a megadott hidratáció és sótartalom az alkalmazott AVPN-tartományba esik-e. A backendben a jelenlegi határok 55–62% hidratáció és 2,5–3,75% só. A két feltétel együttes teljesülése adja az összesített AVPN-jelzőt.

Ellenőrzés	Elfogadott tartomány	Megjelenítés
Hidratáció	55–62%	AVPN-tartomány / kívül
Só	2,5–3,75%	AVPN-tartomány / kívül
Összesített	Mindkettő teljesül	AVPN-kompatibilis tartomány / ellenőrizendő érték

A felület az élesztőmennyiséget kifejezetten becslésként kezeli. Ez fontos szakmai korlát: a tényleges fermentációt a liszt, az élesztő vagy kovász aktivitása és a környezet egyéb tényezői is befolyásolják.

Pizza Maestro Főoldal Kalkulátor Receptek david Képek

DOUGH CALCULATOR

Nápolyi pizzatészta kalkulátor

Az AVPN hivatalos 2024-es iránymutatói alapján jelöljük a hidratáció és sótartalom elfogadott tartományát.

Gombócok száma
4

Gombóc tömege (g)
250

Hidratáció (%)
62
✓ AVPN-tartomány

Só (%)
2.8
✓ AVPN-tartomány

Élesztő / kovász típusa
Friss élesztő

Szobahőmérsékletű fermentáció
Hőmérséklet (°C)
23
Idő (óra)
6

Hűtőben
Hőmérséklet (°C)
4
Idő (óra)
18

> Recept kiszámítása

Kiszámított recept ✓ AVPN-kompatibilis tartomány

1000 g össztészta

Víz	376.2 g	Liszt	606.8 g
Só	28.0 g	Friss élesztő	0.98 g

Hidratáció: 62.0% ✓ 55-62%

Só: 2.8% ✓ 2.5-3.75%

Fermentáció
Szoba: 6 óra / 23 °C - Hűtő: 18 óra / 4 °C - Összesen: 24 óra
Az élesztőmennyiség iránymutatói becslés a megadott hőmérséklet alapján a friss élesztővel való keverés és a környezet jelenlétének befolyásolja a valódi fermentációt.

Recept mentése

Pizza Maestro - oktatási projekt - React + ASP.NET Core + MySQL

3. ábra Recept kiszámítása

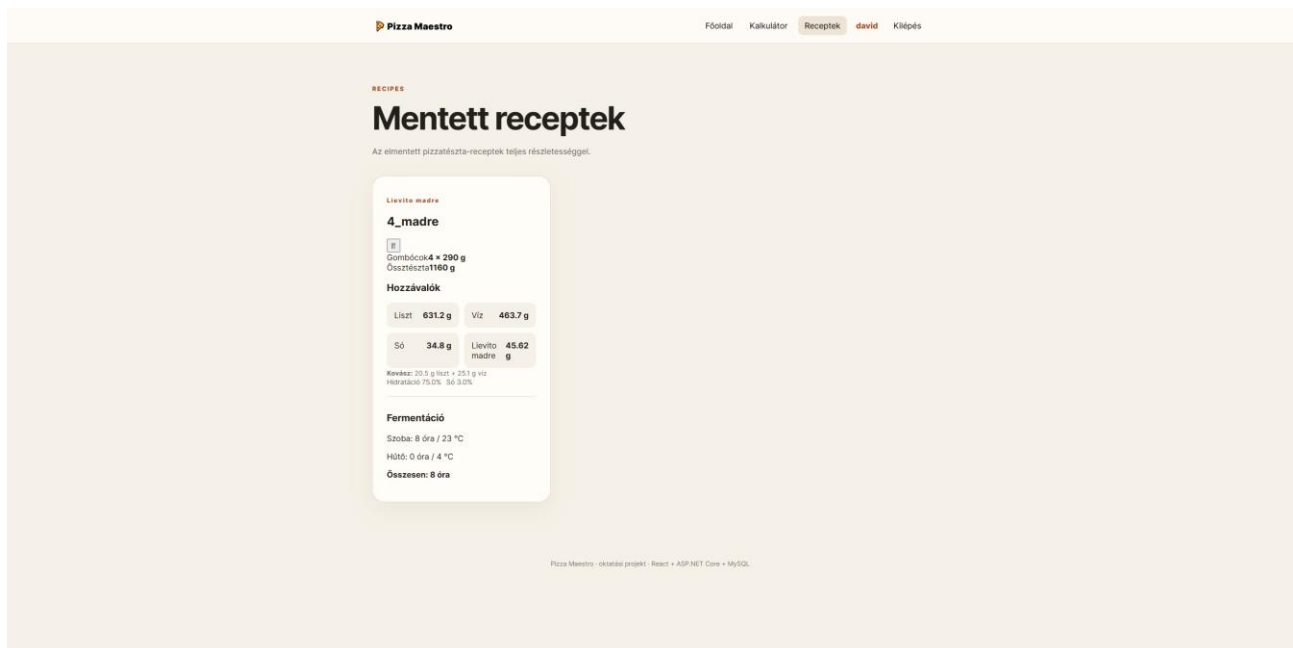
6.4 Mentett receptek

A kiszámított recept név megadásával menthető. A frontend a teljes számítási eredményt elküldi a /api/recipes végpontra, a backend pedig új Recipe entitást hoz létre a bejelentkezett felhasználóhoz rendelve.

A Mentett receptek oldal a recepteket létrehozási idő szerint csökkenő sorrendben kéri le. Minden kártya megjeleníti a recept nevét, a gombócok adatait, az összetésza tömegét, a hozzávalókat, a hidratációt, a sót és a fermentációs adatokat.

A törlés megerősítő ablak után a DELETE /api/recipes/{id} végpontra küldött kérésből történik. A backend csak akkor töröl, ha az adott recept a bejelentkezett felhasználóhoz tartozik.

```
GET    /api/recipes
GET    /api/recipes/{id}
POST   /api/recipes
DELETE /api/recipes/{id}
```



4. ábra Menetett receptek megtekintése

6.5 Reszponzív és felhasználói működés

A frontend komponensekre bontott felépítése lehetővé teszi, hogy a kalkulátor, az eredménykártya, a navigáció és a receptek egymástól függetlenül fejlődjenek. Az App komponens csak az oldalállapotot és a hitelesítési állapotot koordinálja.

A felhasználói élmény szempontjából fontos visszajelzések: számítás közben a kalkuláció gomb tiltódik, receptmentéskor a mentési gomb tiltódik, hiba esetén a kliens érthető üzenetet jelenít meg, a recepttörlés pedig megerősítést kér.

A fejlesztés során több frontend–backend összekötési hibát is javítottunk. A korai változatban az Axios hibás API-alapot használt, ami Network Errorhez vezetett; később az API-cím egységesítésre került, és a jelenlegi kód VITE_API_URL környezeti változót használ, tartalékértékként pedig localhostos HTTPS címet ad.

A fejlesztési környezetben a localhost HTTPS tanúsítvány megbízhatóságának kezelése is szükséges volt, különösen a Swagger elérésekor. Ezek a problémák nem üzleti logikai hibák voltak, hanem a helyi fejlesztési környezet konfigurációjához tartoztak.

The screenshot shows the 'Pizza Maestro' app interface. At the top, there's a navigation bar with the app logo, a 'Főoldal' button, and a 'Kalkulátor' button. Below this, there are links for 'Receptek', 'david', and 'Kilépés'. The main content area is titled 'DOUGH CALCULATOR' and 'Nápolyi pizzatészta kalkulátor'. It includes a descriptive text about AVPN guidelines. Below this, there are five input fields with labels: 'Gombócok száma' (value: 4), 'Gombóc tömege (g)' (value: 250), 'Hidratáció (%)' (value: 62, with a green checkmark and 'AVPN-tartomány' label), 'Só (%)' (value: 2.8, with a green checkmark and 'AVPN-tartomány' label), and 'Élesztő / kovász típusa' (value: 'Erice élesztő').

5. ábra Menetett receptek megtekintése

7. Backend implementáció részletei

A backend ASP.NET Core Web API-ra épül. A Program.cs regisztrálja a controllereket, a Swagger szolgáltatást, a DoughCalculatorService-t, az Entity Framework Core DbContextet, a JWT Bearer hitelesítést, az autorizációt és a CORS szabályt.

```
builder.Services.AddControllers();
builder.Services.AddSwaggerGen();
builder.Services.AddScoped<DoughCalculatorService>();
builder.Services.AddDbContext<AppDbContext>(...);
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(...);
builder.Services.AddAuthorization();
builder.Services.AddCors(...);
```

A PizzaMaestro.API.csproj jelenlegi függőségei között a

Microsoft.AspNetCore.Authentication.JwtBearer 8.0.19, Microsoft.EntityFrameworkCore.Design 8.0.19, Microsoft.Extensions.Identity.Core 8.0.19, Pomelo.EntityFrameworkCore.MySql 8.0.3 és Swashbuckle.AspNetCore 6.6.2 szerepel.

A backend indítási pipeline-ja először a Swagger és CORS szolgáltatásokat állítja be, majd a hitelesítést és autorizációt, végül a controllereket térképezi fel.

7.1 REST API-végpontok

HTTP	Végpont	Funkció	Hitelesítés
POST	/api/auth/register	Felhasználó létrehozása	–
POST	/api/auth/login	Bejelentkezés, JWT kiadása	–
POST	/api/calculator	Pizzatészta kiszámítása	–
GET	/api/recipes	Saját receptek	JWT
GET	/api/recipes/{id}	Saját recept lekérése	JWT
POST	/api/recipes	Recept mentése	JWT
DELETE	/api/recipes/{id}	Saját recept törlése	JWT

A CalculatorController egyetlen POST végpontot biztosít. Alapvető validáció történik a gombócok, gombóctömeg, hidratáció és só értékeire; hibás bemenet esetén BadRequest válasz érkezik.

A RecipesController azonosított felhasználóhoz köti a lekérdezéseket és mentéseket, így a REST API nem csak CRUD-funkciót, hanem tulajdonosi jogosultságot is megvalósít.

7.2 Tésztaszámítási algoritmus

A DoughCalculatorService először az össztészta tömegét számítja ki a gombócok számából és tömegéből. Ebből a százalékos összetétel alapján meghatározza a só, liszt és víz mennyiségét. A liszt számítása a baker's percentage logikáját követi.

```
total = Balls × BallWeight  
salt = total × Salt / 100  
flour = total / (1 + Hydration/100 + Salt/100)  
water = flour × Hydration / 100
```

Kovász használata esetén a service külön kiszámítja a preferment mennyiségét. Folyékony kovásznál 8%, lievito madre esetén 7% lisztarányt alkalmaz; a preferment liszt- és víztartalma bekerül a teljes recept liszt- és vízmennyiségébe.

Típus	Preferment arány	Preferment liszt/víz arány
Folyékony kovász	8%	50% liszt / 50% víz
Lievito madre	7%	45% liszt / 55% víz
Friss élesztő	–	Nincs preferment levonás
Száritott élesztő	–	Nincs preferment levonás

7.3 Élesztő- és fermentációs becslés

A jelenlegi implementáció az élesztőmennyiséget gyakorlati becsléssel számítja. Az algoritmus a szobahőmérsékleti és hűtős időből egy effektív időt, a két szakasz hőmérsékletéből pedig súlyozott átlagos hőmérsékletet képez.

```
hours = max(1, RoomHours + FridgeHours × 0.15)  
temp = weighted average of room/fridge temperatures  
baseFresh = 0.17 × flour(kg) × (24 / hours)  
           × 2((23 - temp) / 8)
```

A számítás korlátozza a kapott értéket egy alsó és felső határ közé. Száritott élesztő esetén a friss élesztőre számolt érték harmadát adja vissza. Kovász esetén a külön preferment-logika lép életbe, ezért az EstimateYeast függvény nulla értékkel tér vissza.

A projekt dokumentáltan nem laboratóriumi fermentációs modell. Ez fontos tervezési döntés: a kalkulátor célja gyakorlati kiindulópont biztosítása, nem a valós fermentáció minden változójának matematikai modellezése.

7.4 Receptkezelés

A RecipesController a kalkulációs szolgáltatást újra felhasználja mentés előtt. Ez megakadályozza, hogy a kliens által elküldött eredményértékek automatikusan megbízható forrásnak minősüljenek: a backend a bemeneti paraméterekből ismét kiszámítja a receptet, majd az így kapott értékeket menti.

```
var calculated = calculator.Calculate(calculatorRequest);

var recipe = new Recipe {
    UserId = userId.Value,
    Name = request.Name.Trim(),
    Balls = request.Balls,
    BallWeight = request.BallWeight,
    Hydration = calculated.Hydration,
    SaltPercent = calculated.SaltPercent,
    FlourGrams = calculated.FlourGrams,
    WaterGrams = calculated.WaterGrams,
    SaltGrams = calculated.SaltGrams,
    YeastGrams = calculated.YeastGrams
};
```

A receptnév kötelező, legfeljebb 200 karakteres lehet. A backend trimeli a nevet és a leírást, majd a mentés CreatedAtAction választ ad. A GET lekérdezések AsNoTracking() használatával készülnek, mivel az olvasási műveleteknél nincs szükség módosításkövetésre.

7.5 Frontend–backend kommunikáció

A kliens API-rétege külön modulokra bontja a hitelesítést, a kalkulációt és a recepteket. Az Axios példány központi helyen állítja be a baseUrl-t és minden kéréshez hozzáadja a localStorage-ban tárolt JWT-t, ha van ilyen.

```
const api = axios.create({
    baseUrl: import.meta.env.VITE_API_URL
    || "https://localhost:5000/api",
    headers: { "Content-Type": "application/json" }
});

api.interceptors.request.use((config) => {
    const token = localStorage.getItem("pizzaMaestroToken");
    if (token) config.headers.Authorization = `Bearer ${token}`;
    return config;
});
```

A calculatorApi a /calculator végpontra küld POST kérést, a recipeApi pedig a receptek GET/POST/DELETE műveleteit kezeli. Ez a szerkezet tisztán elválasztja a UI-komponenseket a HTTP-kommunikáció részleteitől.

8. Biztonsági megoldások

8.1 Jelszókezelés

A backend nem tiszta szöveggént tárolja a jelszavakat. Az ASP.NET Core PasswordHasher<User> használata biztosítja a jelszó hash-elését és ellenőrzését.

8.2 JWT-alapú jogosultság

A védett API-végpontok [Authorize] attribútummal vannak ellátva, a token validálása a Program.cs-ben történik. A kliens által küldött Authorization fejlécben szereplő Bearer token alapján azonosítható a felhasználó.

8.3 Tulajdonosi ellenőrzés

A receptek esetében nem elegendő a hitelesítés ténye: a backend a JWT-ből kiolvasott UserId alapján szűri a rekordokat. Így egy másik felhasználó receptjének lekérése vagy törlése nem lehetséges pusztán az ID ismeretével.

8.4 Éles üzemeltetéshez szükséges megerősítések

A repository aktuális appsettings.json fájlja fejlesztési célú kapcsolat- és JWT-beállításokat tartalmaz. Ezeket éles környezetben környezeti változókbá vagy titkos konfigurációs szolgáltatásba kell áthelyezni, és a repositoryban jelenleg látható hitelesítő értékeket rotálni kell. A dokumentáció szándékosan nem közli ezeket az értékeket.

9. Tesztelés

9.1 Tesztelési stratégia

A Pizza Maestro tesztelése három szinten értelmezhető: számítási logika ellenőrzése, API-szintű funkcionális ellenőrzés és teljes felhasználói folyamatok manuális bejárása. A fejlesztés során a legfontosabb cél az volt, hogy a frontend és backend közötti kapcsolat, a számítás és a hitelesített receptkezelés működőképes legyen.

Terület	Tesztelési cél	Módszer
Kalkuláció	Helyes tézsaértékek	Bemenet → API → eredmény ellenőrzése
Validáció	Hibás értékek elutasítása	Negatív/határértékes kérések
Hitelesítés	JWT kiadás és ellenőrzés	Regisztráció + login + védett endpoint
Receptek	Mentés, listázás, törlés	Teljes CRUD folyamat
Frontend	Megjelenítés és állapotkezelés	Böngészős manuális teszt
Integráció	Frontend–backend kapcsolat	Axios + localhost API

A jelenlegi repositoryban külön automata tesztprojekt nem látható, ezért a dokumentáció nem állít olyan automatizált tesztlefedettséget, amelyet a forráskód nem támaszt alá. A további tesztelési irányokat a 12. fejezet részletezi.

9.2 Manuális teszteset-napló

Teszteset	Bemenet / művelet	Elvárt eredmény	Státusz
Regisztráció	Érvényes név, e-mail, ≥6 karakteres jelszó	Sikeres regisztráció	Sikeres
Duplikált e-mail	Már létező e-mail	409 Conflict	Sikeres
Bejelentkezés	Helyes e-mail + jelszó	JWT token és user objektum	Sikeres
Hibás jelszó	Helytelen jelszó	401 Unauthorized	Sikeres
Kalkuláció	4×250 g, 62%, 2,8%	JSON eredmény és AVPN-jelző	Sikeres
Hibás kalkuláció	0 gombóc / negatív érték	400 Bad Request	Sikeres
Recept mentés	Bejelentkezve, névvel	Recept megjelenik a listában	Sikeres
Saját recept lista	GET /api/recipes	Csak saját receptek	Sikeres
Recept törlés	Saját recept DELETE	204 No Content	Sikeres
Védett endpoint	Token nélkül /api/recipes	401 Unauthorized	Sikeres

A tesztelés során különösen fontos volt az a hibaeset, amikor a frontend elérte a felületet, de az API-hívás Network Errorrel leállt. Ennek kivizsgálása során az Axios baseURL, a backend portja, a HTTPS tanúsítvány és a CORS-beállítás együtt került ellenőrzésre.

9.3 Fejlesztési és hibajavítási folyamat

A projekt fejlesztése nem egyetlen lépésben történt. A működő vizsgaprojekt kialakításához több, egymásra épülő javításra volt szükség.

Probléma / változtatás	Megoldás	Eredmény
EF Core eszköz és projektverzió eltérése	dotnet-ef 8.0.19-re igazítása	Kompatibilis migrációs környezet
Frontend hiányos csomagkonfigurációja	package.json helyreállítása / függőségek rögzítése	Vite/React indítható projekt
Hibás Axios API-cím	localhostos API-alap és VITE_API_URL kialakítása	Frontend–backend kommunikáció helyreállt
HTTPS tanúsítvány figyelmeztetés	Fejlesztői tanúsítvány megbízhatóvá tétele	Swagger és HTTPS API használható
CORS probléma	AllowFrontend policy beállítása	Böngészőből indított API-kérések engedélyezve
Receptmentés finomítása	Backend oldali újraszámítás és user ownership	Megbízhatóbb adatmentés
Hitelesítés bővítése	JWT Bearer + [Authorize]	Védett recept API

A fejlesztési ciklus egyik fontos tanulsága az volt, hogy egy full-stack alkalmazásban a hiba nem feltétlenül az üzleti logikában található. A frontend, a backend, a böngésző biztonsági szabályai, a helyi HTTPS és a NuGet/npm függőségek verziói együttesen határozzák meg, hogy a rendszer ténylegesen működik-e.

10. Telepítési és üzemeltetési útmutató

10.1 Előfeltételek

- .NET 8 SDK
- Node.js és npm
- MySQL szerver
- Git a repository kezeléséhez
- modern böngésző

10.2 Backend indítása

```
cd Backend/PizzaMaestro.API
dotnet restore
dotnet run --urls "https://localhost:7000;http://localhost:5000"
```

Swagger:
<https://localhost:7000/swagger>

10.3 Frontend indítása

```
cd Frontend
npm install
npm run dev
```

A Vite fejlesztői szerver általában saját fejlesztői porton indul; a pontos portot a terminál jelzi. A VITE_API_URL környezeti változóval a frontend API-címe külön konfigurálható.

10.4 Adatbázis konfiguráció

A backend az appsettings.json DefaultConnection beállítását használja. A jelenlegi fejlesztési konfiguráció MySQL localhost szerveret, 3306-os portot és pizzamaestro adatbázist vár. Éles használat esetén a kapcsolati adatokat nem szabad a forráskódban tárolni.

11. Követelmények áttekintése

Követelmény / elvárás	Állapot	Megjegyzés
Frontend technológia használata	Megvalósítva	React + Vite
Backend Web API	Megvalósítva	ASP.NET Core Web API .NET 8
Relációs adatbázis	Megvalósítva	MySQL + EF Core
CRUD jellegű adatkezelés	Megvalósítva	Receptek létrehozása, lekérése, törlése
Felhasználói regisztráció	Megvalósítva	Validáció és jelszóhash
Bejelentkezés	Megvalósítva	JWT token
Jogosultságkezelés	Megvalósítva	[Authorize] + felhasználóazonosítás
Üzleti logika	Megvalósítva	DoughCalculatorService
API dokumentáció	Megvalósítva	Swagger
Reszponzív felület	Megvalósítva	CSS-alapú frontend
Tesztelés	Részben dokumentált	Manuális és integrációs ellenőrzések; automatizálás bővíthető

A repository README-je egyértelműen vizsgaprojektként azonosítja az alkalmazást, és a React + Vite, ASP.NET Core Web API (.NET 8), valamint MySQL + Entity Framework Core technológiai felépítést rögzíti.

12. Továbbfejlesztési lehetőségek

12.1 Automatizált tesztek

Érdemes külön unit teszteket készíteni a DoughCalculatorService számára, különösen a hidratáció, só, élesztő/kovász típus és fermentációs idő határértékeire. API-szintű integrációs tesztekkel a regisztráció, login, kalkuláció és receptkezelés teljes folyamata is automatizálható.

12.2 Konfiguráció és titkok

A kapcsolat-stringet és JWT-kulcsot környezeti változóba vagy secrets-kezelőbe célszerű áthelyezni. A jelenlegi repositoryban látható fejlesztési kulcsok éles környezetben nem használhatók változtatás nélkül.

12.3 Kalkuláció továbbfejlesztése

A jelenlegi élesztőbecslés gyakorlati közelítés. A későbbi verziókban külön fermentációs modell, kovászérettség, lisztparaméterek, dagasztási módszer és célzott fermentációs görbe is figyelembe vehető.

12.4 Felhasználói funkciók

- recept szerkesztése
- recept másolása és újraszámítása
- kedvencek
- recept exportálása PDF-be
- részletes fermentációs idővonal
- több nyelv támogatása

13. Kódminőség és karbantarthatóság

A projekt egyik erőssége a felelősségek elkülönítése. A frontend külön page, component és api rétegeket használ; a backend pedig controller, service, DTO, model és data rétegekre tagolódik. Ez csökkenti annak kockázatát, hogy egyetlen komponensben keveredjen a felület, az adatkezelés és az üzleti logika.

Terület	Megoldás	Karbantarthatósági előny
Üzleti logika	DoughCalculatorService	Önállóan módosítható és tesztelhető
API kommunikáció	api/*.js	UI-tól elkülönülő HTTP réteg
Adatbázis	AppDbContext + EF Core	Erősen típusos ORM
Hitelesítés	AuthController + JWT	Központosított tokenkezelés
Felület	React komponensek	Újrafelhasználhatóság
DTO-k	CalculatorRequest/Response	API-szerződés elkülönítése

A jelenlegi kódbázis további tisztítási lehetősége a validációk egységesítése, a hibaválaszok szabványosítása, a környezeti konfiguráció teljes leválasztása, valamint a tesztek bevezetése. Ezek nem igénylik az alaparchitektúra megváltoztatását.

14. Összegzés

A Pizza Maestro egy teljes stackű, adatbázis-alapú webalkalmazás, amely a nápolyi pizzatészta receptjének kiszámítását és a receptek személyhez kötött kezelését valósítja meg. A rendszer React + Vite kliensből, ASP.NET Core Web API backendből és MySQL adatbázisból áll.

A projekt legfontosabb szakmai elemei a szerveroldali számítás, az Entity Framework Core alapú adatkezelés, a JWT-alapú hitelesítés, a saját receptek tulajdonosi ellenőrzése, a Swagger API-dokumentáció és a komponensalapú frontend.

A fejlesztési folyamat során több olyan problémát is kezeltünk, amelyek tipikusan egy full-stack vizsgaprojektben jelentkeznek: NuGet-eszközverziók összehangolása, hiányos frontend konfiguráció, hibás API-cím, CORS, HTTPS tanúsítvány és frontend–backend integráció. Ezek megoldása után kialakult egy jól elkülönített és továbbfejleszthető alap.

Összességében a Pizza Maestro nem csupán egy matematikai kalkulátor, hanem egy olyan demonstrációs alkalmazás, amelyben a frontendfejlesztés, REST API-tervezés, adatbázis-kezelés, hitelesítés, üzleti logika és üzemeltetési szempontok egyetlen koherens rendszerben jelennek meg.

Forrásjegyzék

- Pizza Maestro GitHub repository — github.com/kurecskocintia/VizsgaRemek
- README.md — projekttechnológiák és indítási útmutató
- ASP.NET Core dokumentáció — Web API és authentication
- Microsoft Entity Framework Core dokumentáció — ORM és DbContext
- Pomelo.EntityFrameworkCore.MySql dokumentáció — MySQL provider
- Swashbuckle / Swagger dokumentáció — OpenAPI felület
- React dokumentáció — komponensek és state kezelés
- Vite dokumentáció — fejlesztői szerver és build
- Axios dokumentáció — HTTP kliens és interceptorok
- AVPN 2024-es szabályzatára hivatkozó projektlogika — hidratációs és sőtartományok

A forráskódra vonatkozó konkrét állításokat a repository aktuális állapotával vetettük össze. A GitHub projekt jelenleg 1 commitot és külön Backend, Frontend és docs könyvtárat mutat, a README pedig a technológiai stack és az indítás fő lépéseit rögzíti.